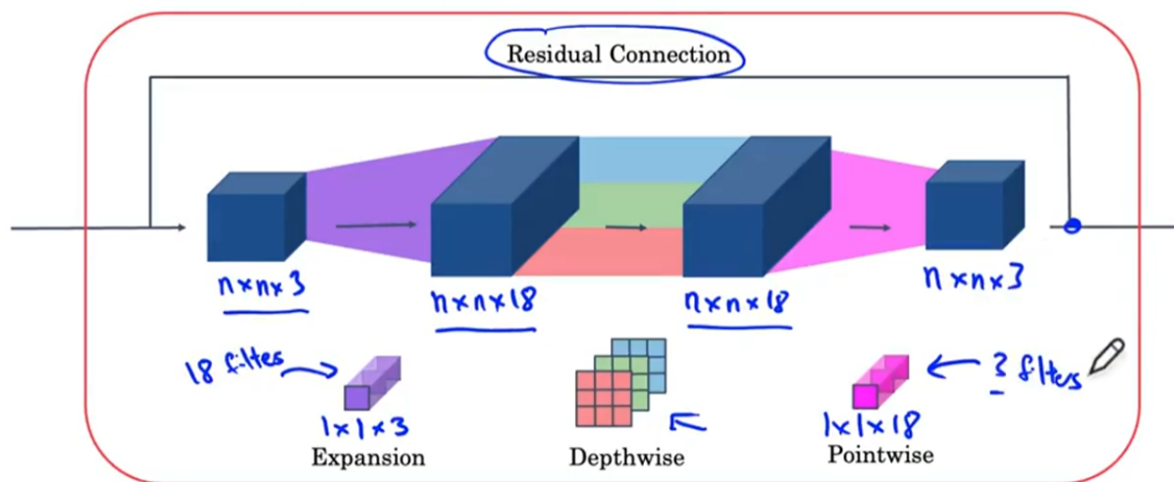


MobileNetV2 Bottleneck

Giải thích từ con số 0 – dành cho người mới bắt đầu

MobileNet v2 Bottleneck



0. Trước tiên: "khối hộp" trong hình là gì?

Trong mạng CNN, dữ liệu được biểu diễn dưới dạng **khối 3 chiều**: **cao** $\times$ **rộng** $\times$ **số kênh**.

- $n \times n \times 3$: một tấm "ảnh" kích thước $n \times n$, mỗi điểm ảnh (pixel) có **3 con số**. Ví dụ ảnh màu có 3 kênh R, G, B.
- $n \times n \times 18$: mỗi pixel có **18 con số**, tức 18 "đặc trưng" khác nhau (cạnh ngang, cạnh dọc, màu đỏ, góc cong...).

Hãy hình dung **mỗi kênh là một tờ giấy**, xếp chồng nhiều tờ lên thì thành khối hộp. Khối càng "dày" thì càng nhiều tờ, tức càng nhiều thông tin mô tả mỗi điểm.

Filter (bộ lọc) là một khối nhỏ trượt trên khối lớn. Tại mỗi vị trí, nó nhân từng số với trọng số rồi cộng lại thành **1 con số**. Có bao nhiêu filter thì đầu ra có bấy nhiêu kênh.

1. Bước 1 - Expansion (Mở rộng): từ 3 lên 18 kênh

Dùng **18 filter**, mỗi filter kích thước **1x1x3**.

- **1x1**: filter chỉ nhìn **đúng 1 pixel**, không nhìn hàng xóm.
- **x3**: nó đọc cả 3 kênh của pixel đó và **trộn** chúng lại.

Ví dụ tính tay: pixel có giá trị $[R, G, B] = [2, 1, 0]$, filter có trọng số $[0.5, 1, -1]$:

$$0.5 \times 2 + 1 \times 1 + (-1) \times 0 = 2$$

Đó là 1 con số cho kênh mới thứ nhất. Làm với 18 filter khác nhau thì ra **18 con số** cho pixel đó. Lặp lại cho mọi pixel, ta được khối $n \times n \times 18$.

Mục đích: "mở rộng không gian làm việc". 3 kênh quá ít để biểu diễn đặc trưng phong phú, nên ta tạm trải ra 18 kênh (gấp **6 lần**, gọi là *expansion factor* $t = 6$) để bước sau có nhiều "nguyên liệu" xử lý.

Sau bước này có hàm kích hoạt **ReLU6**: cắt số âm về 0 và chặn trên ở 6.

2. Bước 2 - Depthwise (Tích chập theo chiều sâu): giữ nguyên 18 kênh

Dùng **18 filter 3×3** , mỗi filter chỉ phụ trách đúng **1 kênh**.

Trong hình, các ô xanh dương, xanh lá, đỏ chồng lên nhau chính là minh họa ý này: **mỗi tờ giấy có một filter riêng**, không tờ nào "nhìn sang" tờ khác.

- **3×3** : lần này filter có nhìn **hàng xóm** xung quanh pixel. Đây là bước học **hình dạng không gian** như cạnh, góc, kết cấu.
- Vì mỗi kênh xử lý riêng, đầu ra vẫn là **$n \times n \times 18$** .

So với tích chập thông thường: filter 3×3 thường có kích thước $3 \times 3 \times 18$, tức nhìn hàng xóm **và** trộn kênh cùng lúc, nên rất tốn phép tính. Depthwise **tách việc ra**: bước này chỉ lo nhìn hàng xóm, việc trộn kênh để bước sau làm.

Sau bước này cũng có **ReLU6**.

3. Bước 3 - Pointwise / Projection (Chiếu): nén từ 18 về 3 kênh

Dùng **3 filter**, mỗi filter kích thước **$1 \times 1 \times 18$** .

Giống bước 1 (chỉ nhìn 1 pixel, trộn các kênh), nhưng **ngược chiều**: từ 18 con số mỗi pixel, trộn lại thành 3 con số. Kết quả là **$n \times n \times 3$** .

Mục đích: nén thông tin về dạng gọn để truyền sang lớp tiếp theo, tiết kiệm bộ nhớ.

Điểm quan trọng: bước này **KHÔNG** có **ReLU**, gọi là *linear bottleneck*.

Lý do: khi chỉ còn 3 kênh (không gian rất hẹp), nếu ReLU cắt bỏ các số âm thì sẽ **mất thông tin không lấy lại được**. Ở 18 kênh thì mất một ít không sao vì còn dư, nhưng ở 3 kênh thì mỗi con số đều quý.

4. Residual Connection (Đường tắt)

Đường kẻ đi vòng phía trên lấy **đầu vào gốc** ($n \times n \times 3$) và **cộng thẳng** vào đầu ra ($n \times n \times 3$):

$$\text{Đầu ra cuối} = \text{Đầu vào} + \text{Kết quả 3 bước}$$

Lý do làm vậy:

- Khối chỉ cần học **phần "chỉnh sửa thêm"** chứ không phải học lại từ đầu, nên dễ học hơn.
- Khi mạng rất sâu, tín hiệu (gradient) có đường tắt để chảy ngược khi huấn luyện, nên không bị "tắt dần".

Điều kiện: phép cộng chỉ làm được khi **hai khối cùng kích thước**. Vì vậy residual chỉ dùng khi đầu vào và đầu ra cùng số kênh và stride = 1 (không thu nhỏ ảnh).

5. Tóm tắt cả khối

Bước	Filter	Nhìn hàng xóm?	Trộn kênh?	Kích thước ra	ReLU6 ?
Expansion	$18 \times (1 \times 1 \times 3)$	Không	Có	$n \times n \times 18$	Có
Depthwise	$18 \times (3 \times 3)$, mỗi cái 1 kênh	Có	Không	$n \times n \times 18$	Có
Pointwise	$3 \times (1 \times 1 \times 18)$	Không	Có	$n \times n \times 3$	Không
Residual	(cộng đầu vào)			$n \times n \times 3$	

Hình dạng khối: hẹp → rộng → rộng → hẹp. Hai đầu nhỏ, giữa phình to, nên gọi là **"Inverted Residual"** (residual đảo ngược). ResNet cổ điển thì ngược lại: rộng → hẹp → rộng.

6. Vì sao thiết kế rắc rối thế? Để rẻ

Đếm số trọng số (tham số) của khối trong hình:

Thành phần	Cách tính	Số tham số
Expansion	$1 \times 1 \times 3 \times 18$	54
Depthwise	$3 \times 3 \times 18$	162
Pointwise	$1 \times 1 \times 18 \times 3$	54
Tổng (MobileNetV2 block)		270
Tích chập thường 3×3 , $18 \rightarrow 18$ kênh	$3 \times 3 \times 18 \times 18$	2.916

Tích chập thường tốn gấp hơn **10 lần**. Đó là lý do MobileNetV2 chạy được trên **điện thoại** (đúng như tên "Mobile"): vẫn học đặc trưng ở không gian rộng 18 kênh, nhưng chỉ tốn chi phí nhỏ.

Hình ảnh ví von để nhớ

Hãy tưởng tượng bạn có một **vali nhỏ** (3 kênh):

1. **Expansion:** mở vali, trải đồ ra **bàn lớn** (18 kênh) để dễ nhìn.
2. **Depthwise:** xem xét **từng món đồ riêng lẻ**, gấp lại cho gọn (mỗi kênh xử lý riêng).
3. **Pointwise:** **xếp lại vào vali nhỏ** một cách khéo léo, cẩn thận không làm rơi đồ (không dùng ReLU).
4. **Residual:** so với vali ban đầu, chỉ ghi nhận **những gì đã thay đổi**.